Q. When we should raise pull request?

-> Ones your task is completed and you want to review your code then only raise pull request.

GIT Internals: About .git Folder

---

1. Who and When '.git' folder get created?

-> When we execute "git init" command or when we execute "git clone" command then these commands itself creating .git folder.

2. Why name start with dot?

-> GIT internally uses Linux inviroment. In Linux, if file name or directory name start with dot then it is considered as hidden file or hidden directory. It is hidden directory so its name start with dot.

3. Why it hidden from user?

-> The .git directory contains the metadata(data about data) about our repository. GIT uses this metadata to manage the repository like track changes, manage versions and handle various repository related operations. By mistake or accidentaly if user modify or delete files of .git directory then GIT will not work propely. So to overcome this problem it is hidden from user.

Key Components of .git folder

-> Some important files and directories of .git folder.

1. HEAD File:

It is normal text file and contains HEAD reference.
HEAD is one kind of pointer and it always point to current branch.
This file tell us currently HEAD is pointing to which branch.

2. config File:

It is normal Text File and contains repository specific configuration settings,such as remote repository URL,user information, branch name,origin name and preferences for the repository.

3. index file:
It is binary file and keeps track of staged files.
It holds the information about the file paths,
file permissions and object ids of staged files.

4. FETCH_HEAD file :

1. When you run the "git fetch" command,Git retrieves the latest changes from a remote repository but does not merge these changes into your local branches. Instead, they are stored in the FETCH_HEAD file.

2. The file typically contains one or more lines, each representing a commit that was fetched from remote. The format of each line is:
<commit-hash> <branch-name> <optional-comment>

Example:
6556f0608ae0cd7ea3709aefedd6e13224172e47 branch 'master' of https://github.com/RRDTNP2024/GIT-PRACTICE

3. If you fetch multiple branches or tags, each fetched reference will have its own entry in FETCH_HEAD.

4. After fetching,If you want to merge fetched changes into your current branch then use 'git merge FETCH_HEAD' command.

5. You can check out the fetched commit using 'git checkout FETCH_HEAD'

6. Note:FETCH_HEAD is a temporary file, and its contents are overwritten each time when you perform a git fetch.
It always represents the most recent fetch operation.

5. ORIG_HEAD File :ORIG_HEAD stores the commit hash of the previous HEAD before certain Git commands are executed.

ORIG_HEAD file is a temporary file, and its contents are overwritten each time when you modify HEAD.it only holds the most recent commit reference

* Summory of HEAD,FETCH_HEAD,ORIG_HEAD pointers

-> HEAD,FETCH_HEAD and ORIG_HEAD acts as a pointers
-> HEAD points to current Branch. i.e HEAD holds current branch reference.
-> FETCH_HEAD points to latest commits fetched from a remote repository
-> ORIG_HEAD points to the previous HEAD before certain operations like rebase, reset, or merge.

6. logs Directory :This directory contains commit history and branch reference history. These logs can be used to recover from mistakes.

7. objects directory:

1. objects directory in the .git folder is the heart of Git's storage mechanism
   It contains all the essential data that Git needs to manage the history and state of your project
2. Internally git store the information in the form of object.
3. In git we have different types of objects like
   a. blob: Actual file content are stored in the form of blob object.
      Blob object holds content without file name or permissions.

   b. tree:tree object store File name,file mode,reference to other objects
   c. commit: commit object holds commit command related information
      like state of the repository at the time of the commit
      parent commit,the author ` s details(who has commited),when commited
      and commit message.

4. All these object are linked together by their hashes.
   Each object is identified by a unique SHA-1 or SHA-256 hash.
   SHA-1 is a 40-character and SHA-256 is 64-character hexadecimal string.

8. refs directory:
1. This directory contains references for branch,tags and remote branch. reference means address.
2. branch reference is store inside head sub-directory.
3. remote branch reference is stored in remote sub-directory
4. tags reference is stored in tags sub-directory.

These references help to manage and keep track of the various branches, tags, and remote-tracking branches within our repository.

9. info directory:

1. This directory Contains additional information and configuration files that are used to manage various aspects of your Git repository

2. exclude file: This file is present in info directory.
This file allows you to specify patterns for files that should be ignored by Git.

3. The 'exclude' file is specific to your local repository and is not shared with others when you push your repository to a remote.

10. hooks directory:

hooks directory contains scripts and that scrips get executed automatically when we fire git commands.

When you initialize a Git repository (git init) or when we clone a Git repository then Git automatically creates a set of sample hook scripts in the hooks directory.

These scripts are typically suffixed with .sample and are inactive by default.

To activate a hook, you remove the .sample suffix and make the script executable

These hooks are used customize and automate Git's behavior, such as enforcing coding standards, validating commit messages, sending notifications, or updating other systems.

Handling the .git Folder:

1. Do Not Modify Manually.Always use Git commands to interact with your repository.

2. Cleanup: Over time, your .git folder might grow in size due to accumulated history. You can use Git commands like 'git gc' (garbage collection) to clean up unnecessary files and optimize the database.